CS204 – Algorithms & Data Structures Laboratory

Dr. V. Vijaya Saradhi,
Dept. Of CSE,
IIT Guwahati
saradhi@iitg.ac.in

Tutorial #03 C++ Classes, Constructors, Destructors

Overview

- Classes
- Member functions
- Default copying
- Access control
- Constructors
- Destructors
- Member access
- Friend class
- Local Classes

Classes

Classes - 01

- A class is a user-defined type.
- It consists of the following members
- data members
- member functions
- type members
- access control

Classes - 02

- Consider the example
- There is no explicit connection between data type and the functions
- Calling `init` function in main as given

```
#include<iostream>
using namespace std;
struct Date
{
  int day;
  int month;
  int year;
};
void init(struct Date &exam_date, int d, int m, int y)
{
  exam_date.day = d;
  exam_date.month = m;
  exam_date.year = y;
};
int main(int argc, char *argv[])
{
  struct Date ed;
  init(ed, 10, 8, 2024);
  cout << "exam date: " << ed.day << "-" << ed.month << "-" << ed.year << endl;
}
```

Classes - 03

- **Members can go un-initialized**

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-class-01b.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
exam date: 32766-0-0
```

```
#include<iostream>
using namespace std;
struct Date
{
  int day;
  int month;
  int year;
};
void init(struct Date &exam_date, int d, int m, int y)
{
  exam_date.day = d;
  exam_date.month = m;
  exam_date.year = y;
};
int main(int argc, char *argv[])
{
  struct Date ed;
  cout << "exam date: " << ed.day << "-" << ed.month << "-" << ed.year << endl;
}
```

Classes - 04

- **Connection between data member of struct Date and functions is established as shown.**

```
#include<iostream>
using namespace std;
struct Date
{
  int day;
  int month;
  int year;
  void init(int d, int m, int y)
  {
    date.day = d;
    m_date.month = m;
    year = y;
  }
};
int main(int argc, char *argv[])
{
  struct Date ed;
  ed.init(10, 8, 2024);
  cout << "exam date: " << ed.day << "-" << ed.month << "-" << ed.year << endl;
}
```

Classes - 05

- Consider the example
- A similar declaration
- Difference is instead of struct, we defined a class
- Direct calling of `init` function in main would result in error
- Accessing members from outside the class is prevented

```cpp
#include<iostream>
using namespace std;
class Date
{
  int day;
  int month;
  int year;
};
void init(Date &exam_date, int d, int m, int y)
{
  exam_date.day = d;
  exam_date.month = m;
  exam_date.year = y;
};
int main(int argc, char *argv[])
{
  Date ed;
  init(ed, 10, 8, 2024);
  cout << "exam date: " << ed.day << "-" << ed.month << "-" << ed.year << end
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-class-03.cc -g
t03-class-03.cc: In function 'void init(Date&, int, int, int)':
t03-class-03.cc:11:12: error: 'int Date::day' is private within this context
  exam_date.day = d;
            ^~~
t03-class-03.cc:5:6: note: declared private here
  int day;
```

Classes - 06

- Members should have public access
- By default, members are all private in class declaration
- Only data members of member functions within the class have access to the private variables
- By default, members are all public in struct declaration
- That is any function or variable outside the scope of the struct can access the data members & member functions

```
#include<iostream>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;
};
void init(Date &exam_date, int d, int m, int y)
{
  exam_date.day = d;
  exam_date.month = m;
  exam_date.year = y;
};
int main(int argc, char *argv[])
{
  Date ed;
  init(ed, 10, 8, 2024);
  cout << "exam date: " << ed.day << "-" << ed.month << "-" << ed.year << endl
}
```

Classes - 07

- Functions declared within a class are called member functions
- Member functions are invoked using specific variable
- In this example, member function `init` is invoked using variable ed

```cpp
#include<iostream>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;

  void init(int d, int m, int y)
  {
    day = d;
    month = m;
    year = y;
  };
};
int main(int argc, char *argv[])
{
  Date ed;
  ed.init(10, 8, 2024);
  cout << "exam date: " << ed.day << "-" << ed.month << "-" << ed.year << endl;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-class-05.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
exam date: 10-8-2024
```

Classes - 08

- Access specifiers
- **Public**: Members declared as public are accessible from outside the class. They can be accessed by any code that has visibility to the class object.
- **Private**: Members declared as private are only accessible within the class itself. They cannot be accessed directly from outside the class.
- **Protected**: Members declared as protected are accessible within the class itself and by classes derived from it. However, for this discussion, we will focus on access within the same class and outside the class, excluding inheritance.

```cpp
#include<iostream>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;

  void init(int d, int m, int y)
  {
    day = d;
    month = m;
    year = y;
  };
};
int main(int argc, char *argv[])
{
  Date ed;
  ed.init(10, 8, 2024);
  cout << "exam date: " << ed.day << "-" << ed.month << "-" << ed.year << endl;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-class-05.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
exam date: 10-8-2024
```

Classes - 09

- Implementing `init` function outside the class scope

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-class-06.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
exam date: 10-8-2024
```

```cpp
#include<iostream>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;
    void init(int, int, int);
};
void Date::init(int d, int m, int y)
  {
    day = d;
    month = m;
    year = y;
  };
int main(int argc, char *argv[])
{
  Date ed;
  ed.init(10, 8, 2024);
  cout << "exam date: " << ed.day << "-" << ed.month << "-" << ed.year << endl;
}
```

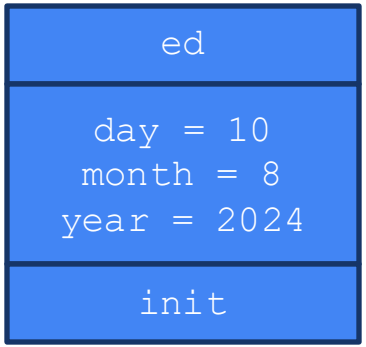
Classes - 10

- Note `day` need not be stated as `Date::day` in the assignment statement
- `day = d;`
- Similarly, `month & year` variables

```
#include<iostream>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;
    void init(int, int, int);
};
void Date::init(int d, int m, int y)
  {
    day = d;
    month = m;
    year = y;
  };
int main(int argc, char *argv[])
{
  Date ed;
  ed.init(10, 8, 2024);
  cout << "exam date: " << ed.day << "-" << ed.month << "-" << ed.year << endl;
}
```

Classes & Objects - 01

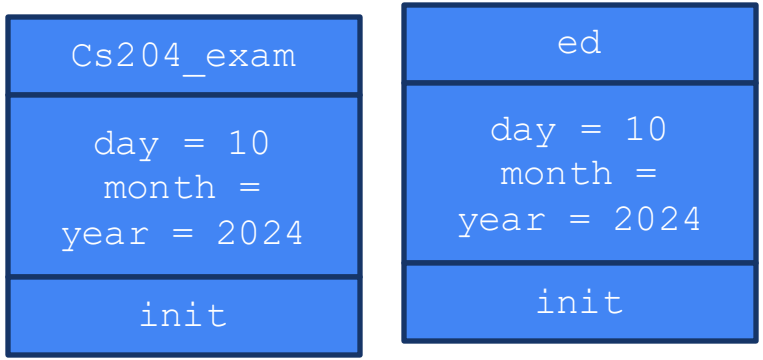
- Definition of "`class Date`" did not cause any memory to be allocated
- Memory is allocated with definition of class object
- Example `ed`
- `ed` is an **object** of type Date




```
#include<iostream>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;
    void init(int, int, int);
};
void Date::init(int d, int m, int y)
  {
    day = d;
    month = m;
    year = y;
  };
int main(int argc, char *argv[])
{
  Date ed;
  ed.init(10, 8, 2024);
  cout << "exam date: " << ed.day << "-" << ed.month << "-" << ed.year << end
}
```

Classes & Objects - 02

- Object assignment – member by member copy
- This is default behavior. This behavior can be changed




```cpp
#include<iostream>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;
    void init(int, int, int);
};
void Date::init(int d, int m, int y)
{
  day = d;
  month = m;
  year = y;
};
int main(int argc, char *argv[])
{
  Date ed;
  ed.init(10, 8, 2024);
  Date cs204_exam = ed;
  cout << "CS204 exam: " << cs204_exam.day << "-" << cs204_exam.month << "-" << cs204_exam.year << en
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-class-07.cc
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
CS204 exam: 10-8-2024
```

Classes & Objects - 03

- Un-initialized object `ed`
- The object is not initialized at the time of declaration
- To initialize `day, month & year` variables of ed object, one has to call `init` function

```cpp
#include<iostream>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;

    void init(int d, int m, int y)
    {
      day = d;
      month = m;
      year = y;
    }
};
int main(int argc, char *argv[])
{
  Date ed;
  cout << "Day: " << ed.day << endl;
  cout << "Month: " << ed.month << endl;
  cout << "Year: " << ed.year << endl;
}
```

```
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ g++ t03-class-08.cc
-g
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ ./a.out
Day: 32693
Month: 1701964084
Year: 32693
```

Classes & Objects - 04

- Initialized object `ed`
- The object is initialized through `init`

```cpp
#include<iostream>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;

    void init(int d, int m, int y)
    {
      day = d;
      month = m;
      year = y;
    }
};
int main(int argc, char *argv[])
{
  Date ed;
  ed.init(10, 8, 2024);
  cout << "Day: " << ed.day << endl;
  cout << "Month: " << ed.month << endl;
  cout << "Year: " << ed.year << endl;
}
```

```
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ g++ t03-class-09.cc
-g
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ ./a.out
Day: 10
Month: 8
Year: 2024
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$
```

Constructors

Constructors - 01

- The use of functions such as `init()` is error prone
- Objects are not initialized and contain garbage values
- Explicit function call must be made every time an object of Date is created
- Constructors are special member functions that are automatically called when an object of the class is created.
- Constructors are used to initialize the class's members.
- Constructor has same name as class name

```
#include<iostream>
using namespace std;
class Date
{
  public:
    int day, month, year;

    Date(int d, int m, int y)
    {
      day = d; month = m; year =y;
    }
    Date()
    {
      day = month = year = 0;
    }
    void print()
    {
      cout << "Day: " << day << endl;
      cout << "Month: " << month << endl;
      cout << "Year: " << year << endl;
    }
};
int main(int argc, char *argv[])
{
  Date ed(10, 8, 2024);
  ed.print();

  Date cs204_ed;
  cs204_ed.print();
}
```

Constructors - 02

- Constructor can be overloaded
- Overloading obey same rules as function overloading rules
- Constructors should differ in their signatures

```
#include<iostream>
using namespace std;
class Date
{
  public:
    int day, month, year;

    Date(int d, int m, int y)
    {
      day = d; month = m; year =y;
    }
    Date()
    {
      day = month = year = 0;
    }
    void print()
    {
      cout << "Day: " << day << endl;
      cout << "Month: " << month << endl;
      cout << "Year: " << year << endl;
    }
};
int main(int argc, char *argv[])
{
  Date ed(10, 8, 2024);
  ed.print();

  Date cs204_ed;
  cs204_ed.print();
}
```

Constructors - 03

- Constructor must have public accessibility

```
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ g++ t03-class-13.cc
-g
t03-class-13.cc: In function ‘int main(int, char**)’:
t03-class-13.cc:25:28: error: ‘Date::Date(int, int, int)’ is private within this
 context
   25 |         Date ed(10, 8, 2024);
      |                            ^
```

```
#include<iostream>
using namespace std;
class Date
{
  public:
    int day, month, year;
    void print()
    {
      cout << "Day: " << day << endl;
      cout << "Month: " << month << endl;
      cout << "Year: " << year << endl;
    }
  private:
    Date(int d, int m, int y)
    {
      day = d; month = m; year =y;
    }
    Date()
    {
      day = month = year = 0;
    }
};
int main(int argc, char *argv[])
{
  Date ed(10, 8, 2024);
  ed.print();

  Date cs204_ed;
  cs204_ed.print();
}
```

Constructors - 04

- Constructor cannot have return statement

```
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ g++ t03-class-14.cc -g
t03-class-14.cc:13:17: error: return type specification for constructor invalid
   13 |                 int Date(int d, int m, int y)
      |                 ^~~
```

```
#include<iostream>
using namespace std;
class Date
{
  public:
    int day, month, year;
    void print()
    {
      cout << "Day: " << day << endl;
      cout << "Month: " << month << endl;
      cout << "Year: " << year << endl;
    }
    int Date(int d, int m, int y)
    {
      day = d; month = m; year =y;
      return 0;
    }
    int Date()
    {
      day = month = year = 0;
      return 0;
    }
};
int main(int argc, char *argv[])
{
  Date ed(10, 8, 2024);
  ed.print();

  Date cs204_ed;
  cs204_ed.print();
}
```

Constructors - 05

- Constructor with member initialization list
- The initialization list is specified with the following syntax

```cpp
#include<iostream>
using namespace std;
class Date
{
  public:
    int day, month, year;
    void print()
    {
      cout << "Day: " << day << endl;
      cout << "Month: " << month << endl;
      cout << "Year: " << year << endl;
    }
    Date(int d, int m, int y) : day(d), month(m), year(y)
    {
      print();
    }
    Date() : day(0), month(0), year(0)
    {
      print();
    }
};
int main(int argc, char *argv[])
{
  Date ed(10, 8, 2024);

  Date cs204_ed;
}
```

```
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ g++ t03-class-15.cc -g
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ ./a.out
Day: 10
Month: 8
Year: 2024
Day: 0
Month: 0
Year: 0
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$
```

Constructors - 07

- Constructors are called under the following circumstances
- **Object Creation**: When objects are created, either by using default or parameterized constructors.
- **Dynamic Allocation**: Constructors are invoked when objects are dynamically created using new.
- **Copying Objects**: The copy constructor is called when an object is copied.
- **Temporary Objects**: Temporary objects, such as those created in expressions or passed by value to functions

Constructors - 08

- **Member Objects**: Constructors of member objects are called before the constructor of the containing class

- **Containers**: When objects are created in containers, constructors are called for each element.

Constructors – 09 – Dynamic Memory

```cpp
#include <iostream>
using namespace std;
class Date
{
public:
    Date()
    {
        cout << "Default constructor called" << endl;
    }
};

int main(int argc, char *argv[])
{
    Date* obj = new Date();
    delete obj;
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-constructor-02.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Default constructor called
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$
```

Constructors – 09 – When object is copied

- Default constructor is NOT called when an object is copied into another object as shown in the code segment below

- We will discuss the cases when constructors are not called

```
Date exam_date(23, 9, 2024);
Date cs204_exam = exam_date;
```

Constructors – 09 – Temporary object creation

```
#include <iostream>
using namespace std;
class Date
{
public:
    Date()
    {
        cout << "Default constructor called" << endl;
    }
    Date(int x)
    {
        cout << "Parameterized constructor called with value: " << x << endl;
    }
};
void func(Date obj)
{
  cout << "In func(Date obj) " << endl;
}
int main(int argc, char *argv[])
{
    func(10);
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-constructor-04.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Parameterized constructor called with value: 10
In func(Date obj)
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$
```

Constructors – 09 – Object is a Member of Another Class

```cpp
#include <iostream>
using namespace std;
class Date
{
public:
    Date()
    {
        cout << "Date constructor called" << endl;
    }
};
class hostel
{
    Date joining_date;
public:
    hostel()
    {
        cout << "hostel constructor called" << endl;
    }
};
int main()
{
    hostel dihing;
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-constructor-05.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Date constructor called
hostel constructor called
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$
```

Constructors – 09 – Object is Created in a Container

```cpp
#include <iostream>
#include <vector>
using namespace std;
class Date
{
public:
    Date()
    {
        cout << "Default constructor called" << endl;
    }
};

int main(int argc, char *argv[])
{
    vector<Date> my_vec(3);
    return 0;
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-constructor-06.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Default constructor called
Default constructor called
Default constructor called
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$
```

Copy Constructor

Copy constructor - 01

- There is one instance in which the constructors provided by the class are NOT invoked
- This is when object is initialized with another object

```
// Constructor is called
Date exam_date(23, 9, 2024);

// Constructor is not called
Date cs204_exam = exam_date;
```

Copy constructor - 02

- There is one instance in which the constructors provided by the class are NOT invoked
- This is when object is initialized with another object

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-class-16.cc
-g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Day :10
Month :8
Year :2024
```

```
#include<iostream>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;
    Date(int d, int m, int y) : day(d), month(m), year(y)
    {
      print();
    }
    void print()
    {
      cout << "Day :" << day << endl;
      cout << "Month :" << month << endl;
      cout << "Year :" << year << endl;
    }
};
int main(int argc, char *argv[])
{
  Date ed(10, 8, 2024);
  Date cs204_exam = ed;
}
```

Copy Constructor - 03

- Initialization of object with another object occurs in three situations
- Explicit initialization of one object with another object

```
Date exam_date(23, 9, 2024);
Date cs204_exam = exam_date;
```

- Passing object as argument to a function

```
void add_years(Date d1, int n);
```

- Return of object as return value of a function

```
Date& add_years(Date &d1, int n);
```

Copy Constructor - 04

- Neither passing a reference argument nor return of a reference results in object initialization

- This is because pass-by-reference does not result in creation of a local copy of the class object

Copy Constructor - 05

```
#include<iostream>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;
    Date(int d, int m, int y) : day(d), month(m), year(y)
    {
      cout << "Design Constructor " << endl;
      print();
    }
    Date(const Date &d1)
    {
      cout << "Copy Constructor " << endl;
      day = d1.day;
      month = d1.month;
      year = d1.year;
      print();
    }
    void print()
    {
      cout << "Day :" << day << endl;
      cout << "Month :" << month << endl;
      cout << "Year :" << year << endl;
      cout << "Year :" << year << endl;
    }
    void print(Date &d)
    {
      cout << "No constructor called" << endl;
      cout << "& Day :" << d.day << endl;
      cout << "& Month :" << d.month << endl;
      cout << "& Year :" << d.year << endl;
    }
};

int main(int argc, char *argv[])
{
  Date ed(10, 8, 2024);
  Date cs204_exam = ed;
  cs204_exam.print(ed);
}
```

Copy Constructor - 06

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-class-17.cc
-g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Design Constructor
Day :10
Month :8
Year :2024
Copy Constructor
Day :10
Month :8
Year :2024
No constructor called
& Day :10
& Month :8
& Year :2024
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$
```

```
      cout << "Year :" << year << endl;
    }
    void print(Date &d)
    {
      cout << "No constructor called" << endl;
      cout << "& Day :" << d.day << endl;
      cout << "& Month :" << d.month << endl;
      cout << "& Year :" << d.year << endl;
    }
};

int main(int argc, char *argv[])
{
  Date ed(10, 8, 2024);
  Date cs204_exam = ed;
  cs204_exam.print(ed);
}
```

Copy Constructor - 07

```
    void print_d(Date d)
    {
      cout << "Copy constructor called???" << endl;
      cout << "& Day :" << d.day << endl;
      cout << "& Month :" << d.month << endl;
      cout << "& Year :" << d.year << endl;
    }
};

int main(int argc, char *argv[])
{
  Date ed(10, 8, 2024);
  Date cs204_exam = ed;
  cs204_exam.print(ed);
  cs204_exam.print_d(ed);
}
```

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03
 -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Design Constructor
Day :10
Month :8
Year :2024
Copy Constructor
Day :10
Month :8
Year :2024
No constructor called
& Day :10
& Month :8
& Year :2024
Copy Constructor
Day :10
Month :8
Year :2024
Copy constructor called???
& Day :10
& Month :8
& Year :2024
```

Copy Constructor - 07

```
Date::Date(const Date &exam_date)
{
  day = exam_date.day;
  month = exam_date.month
  year = exam_date.year
}
```

Destructors

Destructor - 01

- Destructors are special member functions that are automatically called when an object of the class is destroyed.
- The primary purpose of a destructor is to perform cleanup tasks, such as
- Releasing resources that the object may have acquired during its lifetime (e.g., dynamic memory, file handles, etc.).
- Example when a class has a pointer whose memory is created using dynamic memory allocation

Destructor - 02

```
#include<iostream>
#include<string.h>
using namespace std;
class Date
{
  public:
    int day;
    int month;
    int year;
    char *format;
    Date(int d, int m, int y) : day(d), month(m), year(y)
    {
      format = new char[64];
      strcpy(format, "In Design Constructor");
      cout << "Design Constructor " << endl;
      print();
    }
    ~Date()
    {
      cout << "Deleting memory for format" << endl;
```

Terminal

```
      delete[] format;
    }
    void print()
    {
      cout << "Day :" << day << endl;
      cout << "Month :" << month << endl
      cout << "Year :" << year << endl;
      cout << "format :" << format << end
    }
};

int main(int argc, char *argv[])
{
  Date ed(10, 8, 2024);
}
```

Destructor - 03

```
Date::~Date()
{
  delete[] format;
}
```

Destructor - 04

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Design Constructor
Day :10
Month :8
Year :2024
format :In Design Constructor
Deleting memory for format
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$
```

Terminal

```
    delete[] format;
    }
    void print()
    {
      cout << "Day :" << day << endl;
      cout << "Month :" << month << endl
      cout << "Year :" << year << endl;
      cout << "format :" << format << end
    }
};

int main(int argc, char *argv[])
{
  Date ed(10, 8, 2024);
}
```

Destructor - 05

- The destructor is called under the following circumstances
- When an object goes out of scope: destructor is automatically called.
- When a dynamically allocated object is deleted: When you delete an object created using new, the destructor is called to clean up the resources before the memory is deallocated.
- When a temporary object is destroyed
- When a container is called
- When an object is explicitly destroyed using std::destroy_at: utility to explicitly destroy objects without deallocating the memory they occupy. These utilities call the destructor directly.

Destructor – 06 – object going out of scope

```cpp
#include <iostream>
using namespace std;
class Date
{
public:
    Date()
    {
        cout << "Constructor called" << std::endl;
    }
    ~Date()
    {
        cout << "Destructor called" << std::endl;
    }
};
int main(int argc, char *argv[])
{
    {
        Date d1;
    }
    cout << "End of main function" << std::endl;
    return 0;
}
```

Constructor is called at the time of object d1 creation

Destructor is called when scope of d1 is ended

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-destructor-01.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Constructor called
Destructor called
End of main function
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$
```

Destructor – 06 – dynamically allocated object is deleted

```cpp
#include <iostream>
using namespace std;
class Date
{
public:
    Date() {
        cout << "Constructor called" << endl;
    }

    ~Date() {
        cout << "Destructor called" << endl;
    }
};
int main(int argc, char *argv[])
{
    Date* obj = new Date();
    delete obj;
    cout << "End of main function" << endl;
    return 0;
}
```

Constructor is called at the time of object creation

Destructor is called on delete operator call

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-destructor-02.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Constructor called
Destructor called
End of main function
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$
```

Destructor – 06 – For Temporary Objects

```cpp
#include <iostream>
using namespace std;
class Date
{
public:
    Date()
    {
        cout << "Constructor called" << endl;
    }
    ~Date()
    {
        cout << "Destructor called" << endl;
    }
};

Date createObject()
{
    return Date();
}
int main()
{
    Date d1 = createObject();
    cout << "End of main function" << endl;
    return 0;
}
```

Temporary object and its behavior

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-destructor-03.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Constructor called
End of main function
Destructor called
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$
```

Destructor – 06 – For Temporary Objects

```cpp
#include <iostream>
using namespace std;
class Date
{
public:
    Date()
    {
        cout << "Constructor called" << endl;
    }
    ~Date()
    {
        cout << "Destructor called" << endl;
    }
};

Date createObject()
{
  return Date();
}
int main()
{
    createObject();
    cout << "End of main function" << endl;
    return 0;
}
```

Temporary object and its behavior

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-class-03a.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
In createObject function
Constructor called
Destructor called
End of main function
```

Destructor – 06 – When a container is called

```cpp
#include <iostream>
#include <vector>
using namespace std;
class Date
{
public:
    Date()
    {
        cout << "Constructor called" << endl;
    }
    ~Date()
    {
        cout << "Destructor called" << endl;
    }
};

int main()
{
    {
        vector<Date> vec(3);
    }
    cout << "End of main function" << endl;
    return 0;
}
```

Constructor called three times called vector has three Date objects

Similarly, destructor called three times for each of the created object above.

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ g++ t03-destructor-05.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$ ./a.out
Constructor called
Constructor called
Constructor called
Destructor called
Destructor called
Destructor called
End of main function
saradhi@saradhi:~/courses/cs204/july-dec-2024/tutorials/03$
```

Array of Objects

Array Of Objects

```cpp
#include <iostream>
using namespace std;
class Date
{
public:
    Date()
    {
        cout << "Constructor called" << endl;
    }
    ~Date() {
        cout << "Destructor called" << endl;
    }
};
int main(int argc, char *argv[])
{
    Date objArray[3];
    return 0;
}
```

Constructor called three times
Destructor called three times

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/labs/week03$ g++ t03-array-of-objects.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/labs/week03$ ./a.out
Constructor called
Constructor called
Constructor called
Destructor called
Destructor called
Destructor called
saradhi@saradhi:~/courses/cs204/july-dec-2024/labs/week03$
```

Pointers to Class Objects

Pointers to Class Objects

```cpp
#include <iostream>
class Date
{
public:
    Date()
    {
        std::cout << "Constructor called" << std::endl;
    }
    ~Date()
    {
        std::cout << "Destructor called" << std::endl;
    }
};
int main(int argc, char *argv[])
{
    Date* objPtr = new Date();
    delete objPtr;

    Date* objArray = new Date[3];
    delete[] objArray;
    return 0;
}
```

Constructor called
Destructor called

Constructor called three times
Destructor called three times

```
saradhi@saradhi:~/courses/cs204/july-dec-2024/labs/week03$ g++ t03-pointer-to-object.cc -g
saradhi@saradhi:~/courses/cs204/july-dec-2024/labs/week03$ ./a.out
Constructor called
Destructor called
Constructor called
Constructor called
Constructor called
Destructor called
Destructor called
Destructor called
saradhi@saradhi:~/courses/cs204/july-dec-2024/labs/week03$
```

Nested Classes

Nested Classes

```cpp
#include<iostream>
using namespace std;
class Date
{
  private:
    int day;
    int month;
    int year;
  public:
    Date(int d, int m, int y): day(d), month(m), year(y)
    {
      print();
    }
    void print()
    {
      cout << "Day: " << day << endl;
      cout << "Month: " << month << endl;
      cout << "year: " << year << endl;
    }

    class Time
    {
      private:
        int hours;
        int minutes;
        int seconds;
      public:
        Time(int h, int m, int s) : hours(h), minutes(m), seconds(s)
        {
          print();
        }
        void print()
        {
          cout << "Hours: " << hours << endl;
          cout << "Minutes: " << minutes << endl;
          cout << "Seconds: " << seconds << endl;
        }
    };
};
```

```cpp
int main(int argc, char *argv[])
{
  Date cs204_exam(10, 8, 2024);
  Date::Time cs204_time(9, 0, 0);
}
```

```
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ g++ t03-class-17.cc -g
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ ./a.out
Day: 10
Month: 8
year: 2024
Hours: 9
Minutes: 0
Seconds: 0
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$
```

Friend Class

Friend Class - 01

- Information access rules (public, private, protected) are too restrictive.
- The friend mechanism gives nonmembers of the class access to non-public members of a class
- A friend may be a non-member function
- A friend may be entire class

Friend Class – non-member function friend

```cpp
#include<iostream>
using namespace std;
class Date
{
  private:
    int day;
    int month;
    int year;
  public:
    Date(int d, int m, int y) : day(d), month(m), year(y)
    {
      print();
    }
    void print()
    {
      cout << "Day: " << day << endl;
      cout << "Month: " << month << endl;
      cout << "Year: " << year << endl;
    }
    friend void access_members(const Date &);
};
void access_members(const Date& cs204_exam)
{
  cout << "Day: " << cs204_exam.day << endl;
  cout << "Month: " << cs204_exam.month << endl;
  cout << "Year: " << cs204_exam.year << endl;
}
int main(int argc, char *argv[])
{
  Date cs204_exam(10, 8, 2024);
  cs204_exam.print();
  access_members(cs204_exam);
  return 0;
}
```

```
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ g++ t03-friend-01.cc -g
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ ./a.out
Day: 10
Month: 8
Year: 2024
Day: 10
Month: 8
Year: 2024
Day: 10
Month: 8
Year: 2024
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$
```

Friend Class – Entire class is a friend

```cpp
#include <iostream>
using namespace std;
class hostel;
class Date
{
  private:
    int day;
    int month;
    int year;
  public:
    Date(int d, int m, int y) : day(d), month(m), year(y)
    {
        print();
    }
    void print() const
    {
        cout << "Day: " << day << endl;
        cout << "Month: " << month << endl;
        cout << "Year: " << year << endl;
    }
    friend class hostel;
};
class hostel
{
  public:
    void access_date_members(const Date& cs204_exam)
    {
        cout << "Accessing from hostel::access_date_members" << endl;
        cout << "Day: " << cs204_exam.day << endl;
        cout << "Month: " << cs204_exam.month << endl;
        cout << "Year: " << cs204_exam.year << endl;
    }
};
int main(int argc, char* argv[])
{
    Date cs204_exam(10, 8, 2024);
    cs204_exam.print();
    hostel dihing;
    dihing.access_date_members(cs204_exam);
    return 0;
}
```

```
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ g++ t03-friend-02.cc -g
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ ./a.out
Day: 10
Month: 8
Year: 2024
Day: 10
Month: 8
Year: 2024
Accessing from hostel::access_date_members
Day: 10
Month: 8
Year: 2024
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$
```

Local Classes

Local Classes - 01

- Declare a class within a non-member function

```
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$ ./a.out
Day: 10
Month: 8
year: 2024
saradhi@user:~/courses/cs204/july-dec-2024/tutorial/week03$
```

```
#include<iostream>
using namespace std;
int main(int argc, char *argv[])
{
  void display_date();
  display_date();
  return 0;
}

void display_date()
{
  int d = 10, m = 8, y = 2024;
  class Date
  {
    private:
      int day;
      int month;
      int year;
    public:
      Date(int d, int m, int y) : day(d), month(m), year(y)
      {
        print();
      }
      void print()
      {
        cout << "Day: " << day << endl;
        cout << "Month: " << month << endl;
        cout << "year: " << year << endl;
      }
  };

  Date cs204_exam(d, m, y);
}
```

Questions?

Thank You